

Analisis Kerentanan Injeksi SQL dan XSS pada Aplikasi LaporAman serta Mitigasinya

<http://dx.doi.org/10.28932/jutisi.vXiX.X>

Riwayat Artikel

Received: xx Bulan 20xx | Final Revision: xx Bulan 20xx | Accepted: xx Bulan 20xx

Creative Commons License 4.0 (CC BY – NC)



Shofie Ardhya Shafina^{#1}, Nakula Syafa Saputra^{#2}, Putu Novita Darmadewi^{#3}, Della Ayu Nurlaili^{#4}

[#] Program Studi Teknik Informatika, Universitas Negeri Surabaya
Jl. Ketintang, Kec. Gayungan, Kota Surabaya, 60231, Indonesia

¹25051204279@mhs.unesa.ac.id

²25051204374@mhs.unesa.ac.id

³25051204281@mhs.unesa.ac.id

⁴25051204319@mhs.unesa.ac.id

✉ Corresponding author: azissuroni@unesa.ac.id

Abstrak — Aplikasi web pengaduan masyarakat “LaporAman” merupakan sistem sederhana berbasis Python (Flask/FastAPI) dengan basis data SQLite. Aplikasi ini memiliki dua halaman utama, yaitu form pengaduan bagi masyarakat dan dasbor admin untuk melihat laporan. Meskipun fungsionalitasnya sederhana, aplikasi ini rentan terhadap serangan keamanan dasar. Penelitian ini bertujuan untuk menganalisis kerentanan injeksi SQL (SQL Injection) pada aplikasi tersebut serta menyusun strategi mitigasinya. Metode yang digunakan adalah simulasi penetration testing dengan pendekatan IMRaD. Pengujian difokuskan pada form login admin. Hasil penelitian menunjukkan bahwa aplikasi rentan terhadap SQL Injection Authentication Bypass menggunakan payload admin' OR '1'='1' --. Kerentanan ini terjadi karena sistem menggunakan string concatenation langsung pada query SQL tanpa parameterisasi input. Mitigasi yang diusulkan adalah penerapan parameterized queries pada database. Teknik ini terbukti efektif menutup celah keamanan dengan sangat baik. Penelitian ini menekankan pentingnya penerapan secure coding sejak tahap awal pengembangan untuk menghindari risiko serangan sederhana namun berbahaya (security by design).

Kata kunci— SQL Injection; keamanan web; mitigasi kerentanan; authentication bypass; penetration testing.

Analysis of SQL Injection and XSS Vulnerabilities in LapoAman Applications and Mitigation

Abstract — “LaporAman” is a simple web-based community complaint application developed using Python (Flask/FastAPI) and SQLite database. The application consists of two main pages: a complaint submission form for the public and an admin dashboard to view reports. Despite its simple functionality, the application is vulnerable to basic security attacks. This study aims to analyze SQL Injection vulnerabilities in the application and propose mitigation strategies. The research employed a penetration testing simulation using the IMRaD approach. Testing was focused on the admin login form. The results showed that the application is susceptible to SQL Injection Authentication Bypass using the payload admin' OR '1'='1' --. This vulnerability occurs due to direct string concatenation in SQL queries without proper input parameterization. The proposed mitigation is the implementation of parameterized queries. This technique has proven highly effective in closing the security gap. This study highlights the importance

of applying secure coding practices from the early stage of development to prevent simple yet dangerous attacks (security by design).

Keywords— SQL Injection; web security; vulnerability mitigation; authentication bypass; penetration testing.

I. PENDAHULUAN

Era digital telah mengubah paradigma pelayanan publik di Indonesia melalui pemanfaatan aplikasi web sebagai sarana partisipasi masyarakat [1]. Berbagai pemerintah desa dan instansi kini mengadopsi sistem pengaduan masyarakat secara daring untuk meningkatkan transparansi, efisiensi, dan akuntabilitas layanan publik [2]. Namun, di balik kemudahan akses dan partisipasi tersebut, terdapat ancaman serius berupa serangan siber yang dapat membahayakan kerahasiaan, integritas, dan ketersediaan data masyarakat [3]. SQL Injection dan Cross-Site Scripting (XSS) merupakan dua kerentanan yang paling sering dieksploitasi pada website pemerintahan dan aplikasi web pengaduan di Indonesia [4].

Studi komprehensif selama dua puluh dua tahun menunjukkan bahwa XSS masih menjadi salah satu ancaman utama bagi keamanan web modern meskipun telah lama ditemukan [5]. Banyak sistem pengaduan masyarakat masih rentan terhadap SQL Injection karena kurangnya penerapan praktik keamanan yang memadai saat proses pengembangan [6]. Penelitian sebelumnya telah menganalisis kerentanan tersebut menggunakan OWASP ZAP dan SQLMap [7]. Audit keamanan website pemerintahan menemukan tingginya kerentanan XSS dan SQL Injection [8]. Analisis keamanan pada website berbasis WordPress menunjukkan pentingnya penggunaan crawling method untuk mendeteksi celah keamanan [9]. Penetration testing pada sistem pelayanan publik semakin diperlukan [10], sementara penerapan Web Application Firewall serta pendekatan machine learning untuk deteksi serangan web juga mulai banyak dikembangkan [11].

Meskipun telah banyak penelitian tentang deteksi dan mitigasi serangan web [12], masih sedikit studi yang secara spesifik fokus pada aplikasi pengaduan masyarakat berskala kecil dengan arsitektur sederhana berbasis Python (Flask/FastAPI) dan basis data SQLite [13]. Kebanyakan penelitian sebelumnya lebih berorientasi pada sistem skala besar atau hanya melakukan pengujian black-box tanpa menyajikan contoh kode rentan beserta implementasi mitigasi secara rinci dan praktis [14]. Hal ini menyebabkan masih adanya kesenjangan antara teori keamanan web dengan praktik pengembangan aplikasi sederhana yang banyak digunakan di tingkat desa dan komunitas [15].

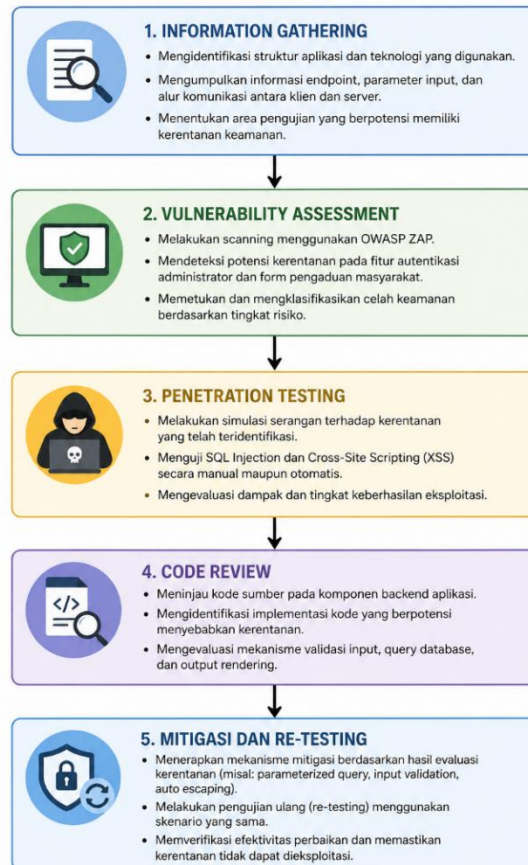
Penelitian ini bertujuan untuk menganalisis kerentanan SQL Injection dan XSS pada web pengaduan masyarakat “LaporAman” serta menyusun strategi mitigasi yang efektif dan praktis [16]. Melalui pendekatan simulasi penetration testing dan analisis kode sumber secara mendalam, penelitian ini diharapkan dapat memberikan rekomendasi konkret bagi pengembang aplikasi serupa agar menerapkan prinsip *security by design* sejak tahap awal pengembangan, sehingga tercipta sistem pengaduan masyarakat yang lebih aman dan dapat diandalkan.

II. METODE PENELITIAN

Penelitian ini menggunakan pendekatan Vulnerability Assessment and Penetration Testing (VAPT) untuk menganalisis kerentanan keamanan pada aplikasi web pengaduan masyarakat LaporAman. Pendekatan VAPT dipilih karena mampu mengintegrasikan proses identifikasi kerentanan secara sistematis dengan simulasi serangan yang menyerupai kondisi nyata, sehingga menghasilkan evaluasi keamanan yang lebih komprehensif, objektif, dan terukur [1]. Melalui pendekatan ini, penelitian tidak hanya berfokus pada pendeteksian celah keamanan, tetapi juga mengevaluasi tingkat eksploitabilitas serta dampak yang dapat ditimbulkan apabila kerentanan tersebut dimanfaatkan oleh pihak yang tidak berwenang.

Tahapan Vulnerability Assessment digunakan untuk mengidentifikasi potensi kerentanan pada aplikasi melalui proses pemetaan fitur, analisis permukaan serangan (attack surface), serta pemindaian keamanan menggunakan berbagai perangkat pengujian. Selanjutnya, tahapan Penetration Testing dilakukan untuk memverifikasi temuan kerentanan dengan mensimulasikan serangan secara terkendali terhadap komponen aplikasi yang dianggap rentan. Pendekatan ini memungkinkan peneliti memperoleh gambaran yang lebih akurat mengenai risiko keamanan yang sebenarnya, termasuk kemungkinan terjadinya akses tidak sah, manipulasi data, maupun penyalahgunaan fungsi sistem.

Dalam penelitian ini, fokus pengujian diarahkan pada kerentanan SQL Injection dan Cross-Site Scripting (XSS) karena kedua jenis serangan tersebut termasuk dalam kategori kerentanan aplikasi web yang paling umum ditemukan dan memiliki tingkat risiko yang tinggi terhadap kerahasiaan, integritas, serta ketersediaan data. Hasil identifikasi dan pengujian kemudian dianalisis melalui proses code review untuk menemukan akar penyebab kerentanan pada tingkat implementasi kode. Berdasarkan hasil analisis tersebut, dilakukan penerapan mekanisme mitigasi keamanan yang sesuai, kemudian dilanjutkan dengan re-testing untuk memverifikasi efektivitas perbaikan yang telah diterapkan. Dengan demikian, pendekatan VAPT memungkinkan proses evaluasi keamanan dilakukan secara menyeluruh mulai dari identifikasi kerentanan, simulasi eksploitasi, implementasi mitigasi, hingga validasi hasil perbaikan keamanan.



Gambar 1. Flowchart Tahapan Penelitian Menggunakan VAPT

A. Desain Penelitian

Penelitian ini merupakan penelitian eksperimental pada bidang keamanan aplikasi web dengan pendekatan *black-box penetration testing*. Pendekatan ini digunakan untuk mensimulasikan serangan dari sisi pengguna tanpa akses awal terhadap konfigurasi internal maupun kode sumber aplikasi. Fokus penelitian diarahkan pada identifikasi dan evaluasi kerentanan SQL Injection serta *Cross-Site Scripting* (XSS), karena kedua jenis serangan tersebut termasuk kerentanan yang paling umum ditemukan pada aplikasi web berbasis input pengguna. Pengujian dilakukan terhadap fitur autentikasi administrator dan form pengaduan masyarakat untuk mengevaluasi mekanisme validasi input, pengolahan data pengguna, serta keamanan interaksi aplikasi dengan basis data.

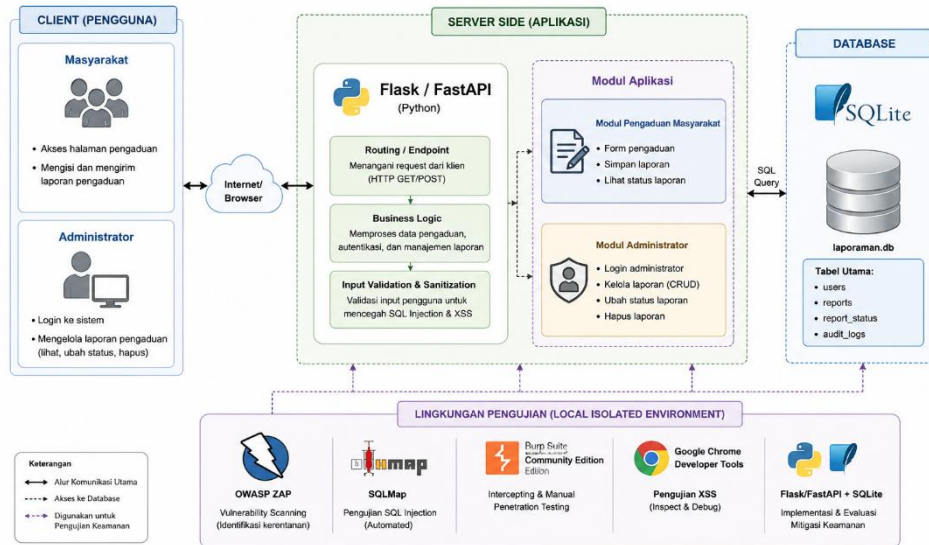
Metodologi *Vulnerability Assessment and Penetration Testing* (VAPT) diterapkan melalui beberapa tahapan utama, yaitu identifikasi kerentanan (*vulnerability assessment*), simulasi serangan (*penetration testing*), analisis kode (*code review*), implementasi mitigasi keamanan, dan pengujian ulang (*re-testing*). Tahap *vulnerability assessment* dilakukan untuk mendeteksi potensi celah keamanan pada aplikasi, sedangkan tahap *penetration testing* dilakukan untuk mengevaluasi kemungkinan eksploitasi terhadap kerentanan yang ditemukan. Selanjutnya, proses *code review* digunakan untuk menganalisis implementasi kode program yang berkaitan dengan pengolahan input dan interaksi basis data. Setelah mitigasi keamanan diterapkan, dilakukan *re-testing* menggunakan skenario pengujian yang sama untuk memverifikasi efektivitas perbaikan keamanan pada aplikasi web.

B. Lingkungan dan Objek Pengujian

Penelitian ini merupakan penelitian eksperimental pada bidang keamanan aplikasi web dengan pendekatan *black-box penetration testing*. Pengujian dilakukan dari sisi pengguna tanpa akses awal terhadap konfigurasi internal sistem guna mensimulasikan skenario serangan nyata pada aplikasi web. Fokus penelitian diarahkan pada identifikasi dan evaluasi kerentanan SQL Injection serta *Cross-Site Scripting* (XSS) yang berpotensi mengganggu kerahasiaan, integritas, dan keamanan data pada aplikasi pengaduan masyarakat.

Metodologi *Vulnerability Assessment and Penetration Testing* (VAPT) diterapkan melalui beberapa tahapan utama, yaitu identifikasi kerentanan (*vulnerability assessment*), simulasi serangan (*penetration testing*), analisis kode (*code review*),

implementasi mitigasi keamanan, dan pengujian ulang (*re-testing*). Rangkaian tahapan tersebut dilakukan untuk mengevaluasi tingkat keamanan aplikasi, mengidentifikasi potensi celah keamanan, serta memverifikasi efektivitas mekanisme mitigasi yang diterapkan terhadap kerentanan yang ditemukan.



Gambar 2. Diagram Arsitektur Aplikasi LaporAman

C. Skenario dan Teknik Pengujian

Objek penelitian dalam studi ini adalah aplikasi web pengaduan masyarakat bernama *LaporAman*. Aplikasi dikembangkan menggunakan bahasa pemrograman Python dengan framework Flask/FastAPI dan basis data SQLite. Sistem terdiri atas dua modul utama, yaitu halaman pengaduan masyarakat dan halaman administrator untuk pengelolaan laporan pengaduan [2]. Arsitektur aplikasi dirancang secara sederhana untuk merepresentasikan karakteristik aplikasi layanan publik berskala kecil hingga menengah yang umum digunakan pada lingkungan desa maupun instansi lokal. Seluruh pengujian dilakukan pada lingkungan lokal (*local isolated environment*) guna menghindari dampak terhadap sistem produksi maupun pihak eksternal.

Penelitian memanfaatkan beberapa tools keamanan sebagai berikut:

1. OWASP Zed Attack Proxy (ZAP) digunakan untuk melakukan *vulnerability scanning* dan identifikasi awal terhadap potensi kerentanan keamanan pada aplikasi web;
2. SQLMap digunakan untuk melakukan pengujian SQL Injection secara otomatis guna mengevaluasi kemungkinan eksploitasi pada interaksi basis data;
3. Burp Suite Community Edition digunakan untuk proses *intercepting*, manipulasi request, dan pengujian penetrasi secara manual;
4. Google Chrome Developer Tools digunakan untuk pengujian *Cross-Site Scripting* (XSS), khususnya dalam mengamati eksekusi payload JavaScript pada sisi klien;
5. Lingkungan pengembangan Flask/FastAPI dan SQLite digunakan dalam proses implementasi mitigasi keamanan serta evaluasi efektivitas perbaikan yang diterapkan pada aplikasi.

TABEL 1
SKENARIO PENGUJIAN KERENTANAN SQL INJECTION DAN XSS

Jenis Pengujian	Target	Teknik	Tujuan
SQL injection	Form login	Authentication bypass	Menguji validasi query
Stored XSS	Form pengaduan	Script injection	Menguji output rendering

D. Tahapan Vulnerability Assessment and Penetration Testing

Tahapan *Vulnerability Assessment and Penetration Testing* (VAPT) pada penelitian ini dilakukan secara sistematis untuk mengidentifikasi, mengevaluasi, dan memverifikasi kerentanan keamanan pada aplikasi web *LaporAman*. Setiap tahapan dirancang untuk merepresentasikan proses pengujian keamanan aplikasi web secara nyata, mulai dari identifikasi area rentan hingga penerapan mitigasi keamanan. Proses pengujian dilakukan secara bertahap agar setiap kerentanan yang ditemukan dapat dianalisis secara lebih terstruktur dan mendalam.

- 1) Information Gathering

Tahap awal dilakukan dengan mengidentifikasi struktur aplikasi, endpoint, parameter input, serta alur komunikasi antara klien dan server. Proses ini bertujuan untuk memahami mekanisme kerja aplikasi dan menentukan area pengujian yang berpotensi memiliki kerentanan keamanan. Selain itu, tahap ini juga digunakan untuk mengidentifikasi fitur yang menerima input pengguna secara langsung, seperti form login administrator dan form pengaduan masyarakat.

2) Vulnerability Assessment

Proses vulnerability assessment dilakukan menggunakan OWASP Zed Attack Proxy (ZAP) untuk mendeteksi potensi kerentanan pada aplikasi web. Pengujian difokuskan pada fitur autentikasi administrator dan form pengaduan masyarakat guna memperoleh pemetaan awal terhadap celah keamanan aplikasi. Tahap ini membantu mengidentifikasi kemungkinan kerentanan yang dapat dieksploitasi pada tahap pengujian berikutnya.

3) Penetration Testing

Tahap penetration testing dilakukan melalui simulasi serangan terhadap kerentanan yang telah teridentifikasi menggunakan teknik SQL Injection dan Cross-Site Scripting (XSS). Pengujian dilakukan secara manual maupun otomatis untuk mengevaluasi tingkat keamanan aplikasi terhadap manipulasi input pengguna dan eksekusi payload berbahaya. Tahap ini bertujuan untuk mengetahui dampak keamanan yang dapat ditimbulkan apabila kerentanan berhasil dieksploitasi oleh penyerang.

4) Code Review

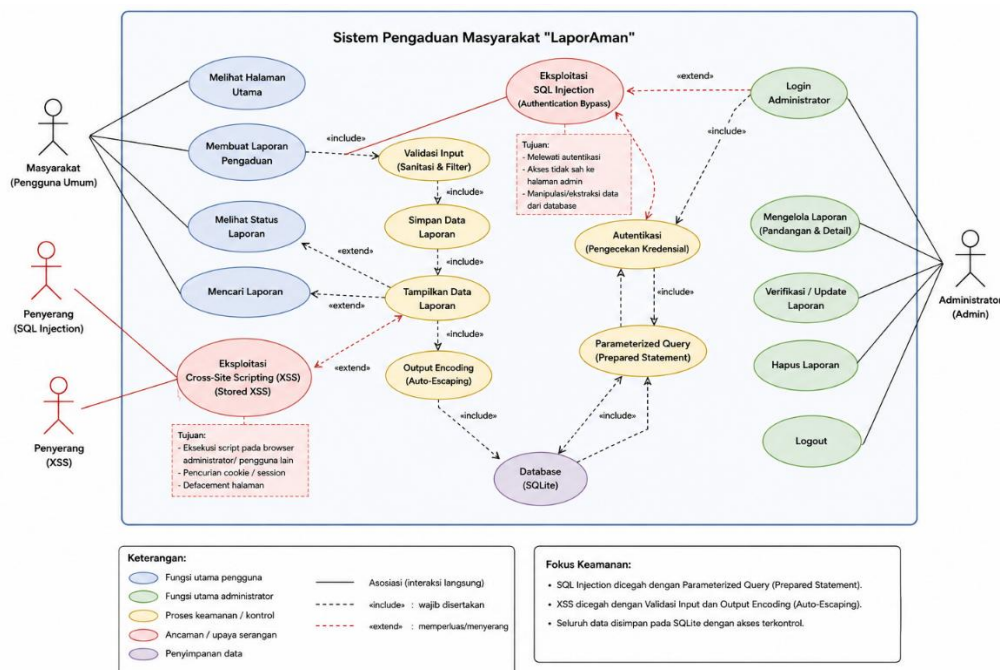
Tahap code review dilakukan terhadap komponen backend aplikasi guna mengidentifikasi implementasi kode yang berpotensi menyebabkan kerentanan keamanan, khususnya pada proses pengolahan input pengguna dan interaksi basis data. Analisis kode dilakukan untuk mengevaluasi penerapan mekanisme validasi input, pengelolaan query basis data, serta mekanisme output rendering pada aplikasi web.

5) Mitigasi dan Re-Testing

Tahap mitigasi dilakukan dengan menerapkan mekanisme keamanan berdasarkan hasil evaluasi kerentanan yang ditemukan selama proses pengujian. Setelah implementasi mitigasi dilakukan, tahap re-testing dilaksanakan menggunakan skenario pengujian yang sama untuk memverifikasi efektivitas perbaikan keamanan yang diterapkan. Tahap ini bertujuan memastikan bahwa kerentanan yang sebelumnya teridentifikasi tidak lagi dapat dieksploitasi pada aplikasi.

E. Teknik Analisis Data

Analisis data dilakukan secara deskriptif dengan mengevaluasi hasil vulnerability scanning, keberhasilan simulasi serangan, serta efektivitas mitigasi keamanan yang diterapkan. Hasil pengujian kemudian diklasifikasikan berdasarkan jenis kerentanan, tingkat risiko, dan dampaknya terhadap keamanan aplikasi web.



Gambar 3. Use Case Diagram Analisis Kerentanan SQL Injection dan XSS pada Aplikasi LaporAman

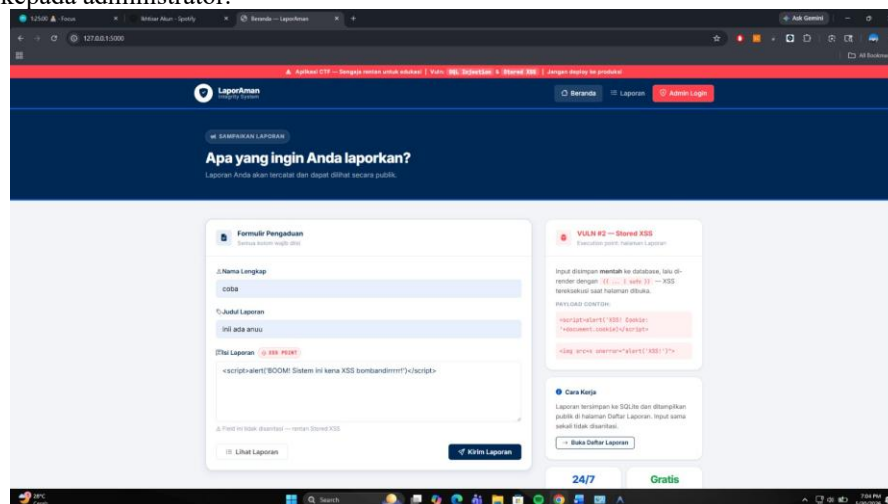
Use case diagram keamanan pada aplikasi LaporAman menggambarkan interaksi antara pengguna, administrator, dan potensi penyerang terhadap sistem. Dari sisi keamanan, terdapat dua ancaman utama yang dianalisis, yaitu SQL Injection pada proses autentikasi administrator dan Stored Cross-Site Scripting (XSS) pada fitur pengaduan masyarakat. SQL Injection menargetkan mekanisme login dengan memanipulasi query database untuk memperoleh akses tidak sah, sedangkan XSS memanfaatkan input laporan yang tidak disanitasi untuk menyisipkan skrip berbahaya yang akan dieksekusi saat laporan ditampilkan. Untuk memitigasi ancaman tersebut, sistem menerapkan validasi input, output encoding (auto-escaping) pada proses penampilan data, serta parameterized query (prepared statement) pada akses database. Dengan penerapan mekanisme keamanan tersebut, integritas data, kerahasiaan informasi, dan keamanan proses autentikasi dapat terjaga dari upaya eksploitasi yang dilakukan oleh penyerang.

III. HASIL DAN PEMBAHASAN

Penelitian ini menemukan bahwa aplikasi web pengaduan masyarakat LaporAman versi awal mengandung kerentanan SQL Injection dan Cross-Site Scripting (XSS) yang signifikan. Pengujian menggunakan OWASP ZAP berhasil mendeteksi 12 alert kerentanan dengan tingkat risiko High dan Medium. Kerentanan paling kritis terdapat pada endpoint pengaduan dan halaman autentikasi administrator. Hasil vulnerability scanning menunjukkan bahwa aplikasi tidak melakukan sanitasi input dengan baik, sehingga memungkinkan penyerang menyisipkan perintah berbahaya [16].

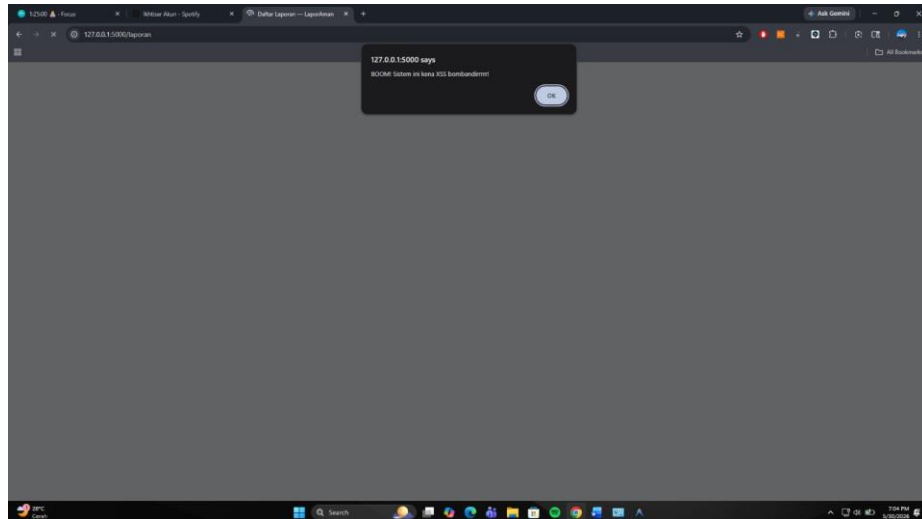
Pada tahap *penetration testing*, serangan SQL Injection berhasil dieksploitasi pada *form* pengaduan. Dengan memasukkan *payload* `admin' OR '1'='1 --` pada *field* *username*, penyerang dapat melewati mekanisme autentikasi dan memperoleh akses penuh sebagai administrator. Hal ini menunjukkan bahwa *query database* dibangun secara dinamis tanpa *prepared statement*, sehingga sangat rentan terhadap serangan injeksi.

Pengujian terhadap fitur pengaduan masyarakat menunjukkan adanya kerentanan Stored Cross-Site Scripting (XSS). Kerentanan ini ditemukan pada *field* "Isi Laporan" yang menerima masukan pengguna tanpa proses sanitasi yang memadai. Penyerang dapat menyisipkan *payload* JavaScript yang kemudian tersimpan di basis data dan dieksekusi kembali ketika laporan ditampilkan kepada administrator.



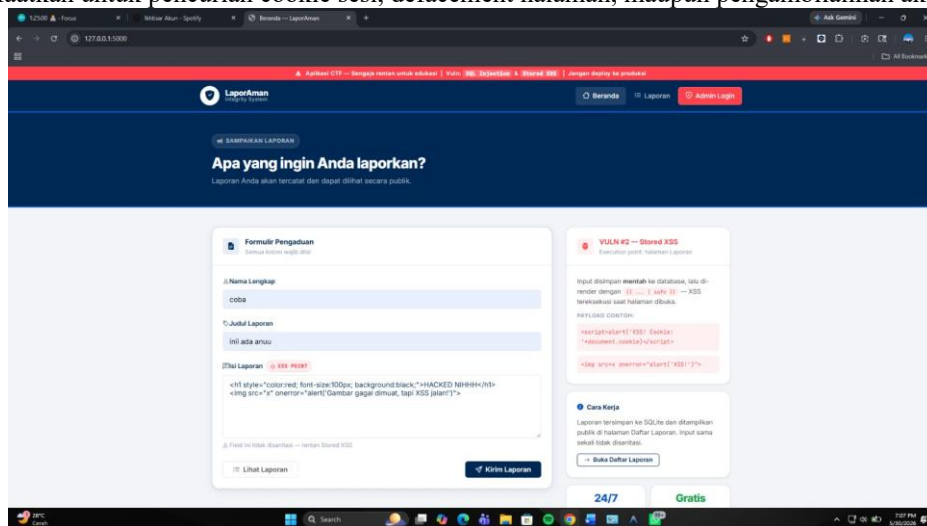
Gambar 4. Injeksi *payload* Stored XSS berupa *script pop-up alert* pada kolom 'Isi Laporan' di formulir pengaduan.

Gambar 3 menunjukkan halaman form pengaduan yang menjadi titik masuk serangan Stored XSS. Pada kondisi ini, sistem menerima dan menyimpan *payload* berbahaya tanpa melakukan validasi maupun sanitasi terhadap input pengguna. Kondisi tersebut memungkinkan skrip berbahaya tersimpan secara permanen pada basis data aplikasi.



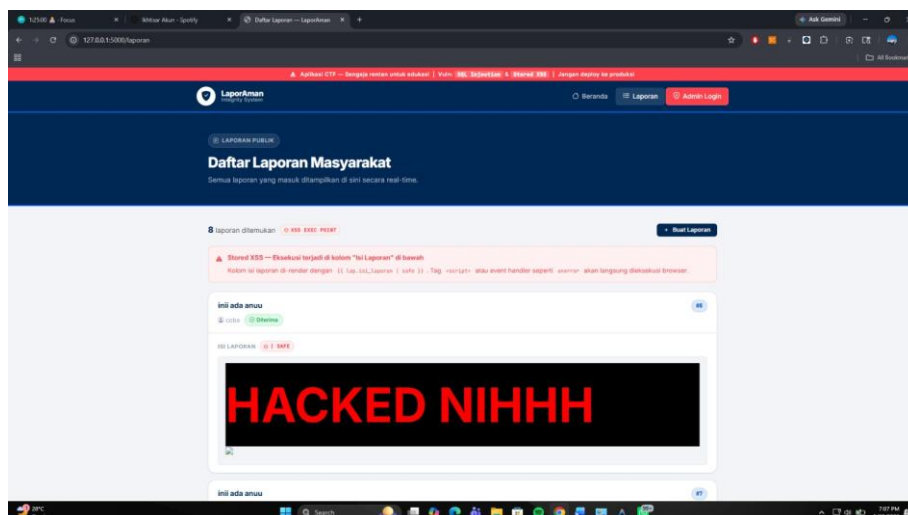
Gambar 5. Hasil eksekusi *payload* *Stored XSS* yang memunculkan *pop-up alert* ketika halaman daftar laporan diakses.

Lebih lanjut, kerentanan *Stored XSS* ini juga dapat dimanfaatkan untuk melakukan *defacement* (perubahan tampilan halaman web secara tidak sah). Penyerang dapat menyisipkan *tag* HTML dan CSS khusus pada *field* input yang tidak disanitasi untuk memanipulasi *interface* web. Gambar 4 memperlihatkan hasil eksploitasi *Stored XSS* ketika laporan ditampilkan pada halaman administrator. *Payload* JavaScript yang sebelumnya disimpan berhasil dieksekusi oleh peramban sehingga membuktikan bahwa mekanisme output rendering tidak menerapkan escaping secara memadai. Kerentanan ini berpotensi dimanfaatkan untuk pencurian cookie sesi, *defacement* halaman, maupun pengambilalihan akun administrator.



Gambar 6. Injeksi *payload* *Stored XSS* tingkat lanjut menggunakan *tag* HTML untuk melakukan *defacement*.

Hasil dari eksploitasi tersebut dapat dilihat pada Gambar 6, di mana *payload* *XSS* berhasil mengubah dan merusak tata letak Halaman Daftar Laporan Masyarakat (terkena *defacement*), yang berpotensi menurunkan tingkat kepercayaan masyarakat terhadap sistem pengaduan. Gambar 5 menunjukkan proses pengujian SQL Injection pada halaman login administrator menggunakan *payload* 'OR '1'='1' --. *Payload* tersebut dimasukkan pada *field* username untuk menguji apakah sistem melakukan validasi dan pemrosesan input secara aman. Hasil pengujian mengindikasikan bahwa aplikasi tidak melakukan parameterisasi query sehingga input pengguna dapat memengaruhi struktur perintah SQL yang dikirim ke basis data.



Gambar 7. Halaman Daftar Laporan Masyarakat yang berhasil dimanipulasi (*defacement*) akibat eksekusi *payload* XSS.

Analisis *code review* mengungkap bahwa kerentanan tersebut disebabkan oleh penggunaan *string concatenation* pada pembuatan *query* SQLite dan *rendering output* tanpa melakukan *escaping*. Framework Flask yang digunakan tidak menerapkan *auto-escaping* secara default pada *template* Jinja2 untuk variabel yang di-render. Gambar 6 memperlihatkan keberhasilan serangan SQL Injection dalam melakukan authentication bypass. Setelah *payload* dieksekusi, sistem memberikan akses ke halaman administrator tanpa memerlukan kredensial yang valid. Kondisi ini menunjukkan bahwa mekanisme autentikasi rentan terhadap manipulasi logika query, sehingga penyerang dapat memperoleh hak akses tidak sah ke dalam sistem. Kerentanan tersebut berpotensi menyebabkan kebocoran data, perubahan informasi, maupun penyalahgunaan fungsi administratif aplikasi.

```
34 <code class="danger">{{ '{{' }} lap.isi_laporan | safe {{ '{{' }} }}</code>.
```

Gambar 8. Potongan kode *template* Jinja2 sebelum mitigasi yang menggunakan filter.

Gambar 7 menunjukkan penggunaan filter `|safe` pada proses rendering data laporan. Penggunaan filter tersebut menyebabkan seluruh konten yang berasal dari pengguna ditampilkan tanpa proses encoding sehingga *payload* HTML maupun JavaScript dapat dieksekusi oleh browser.

```
34 <code class="danger">{{ '{{' }} lap.isi_laporan| {{ '{{' }} }}</code>.
```

Gambar 9. Potongan kode *template* Jinja2 setelah mitigasi dengan menghapus filter

Sebagaimana ditunjukkan pada Gambar 8, filter `|safe` dihapus sehingga Jinja2 kembali menerapkan mekanisme *auto-escaping* secara otomatis. Dengan pendekatan ini, karakter khusus akan dikonversi menjadi entitas HTML yang aman sehingga *payload* yang tersimpan hanya ditampilkan sebagai teks biasa dan tidak dapat dieksekusi.

```
136 query = f"SELECT * FROM users WHERE username='{username}'"
137 user = db_query_raw(query, fetchone=True)
```

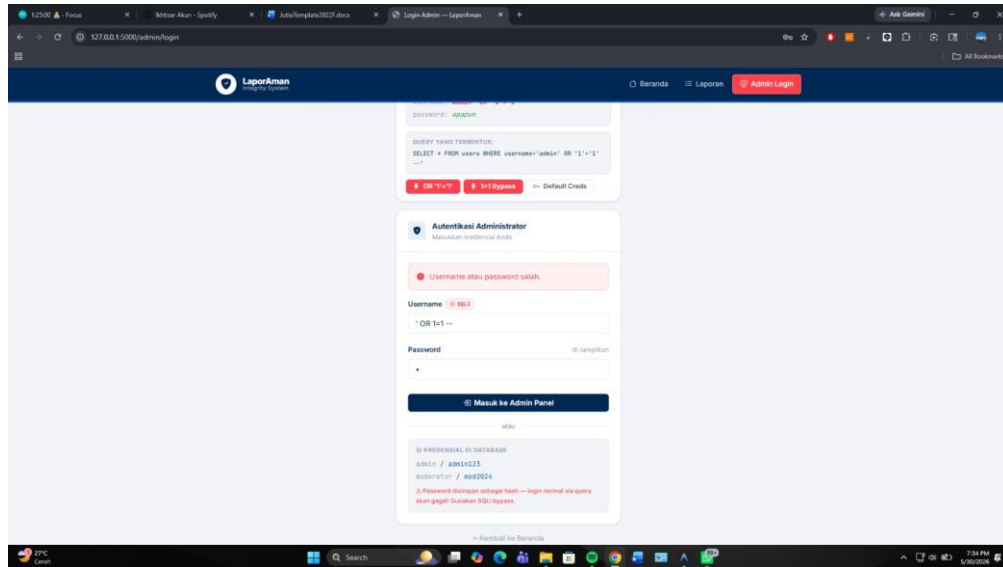
Gambar 10. Potongan kode *query* SQLite sebelum mitigasi yang rentan terhadap injeksi akibat penggunaan *f-string*.

Gambar 9 menunjukkan implementasi kueri yang rentan karena menggunakan teknik *string concatenation* untuk menggabungkan input pengguna ke dalam perintah SQL. Pendekatan tersebut memungkinkan penyerang memodifikasi struktur kueri melalui *payload* SQL Injection.

```
query = f"SELECT * FROM users WHERE username={PH}"
user = db_query(query, (username,), fetchone=True)
```

Gambar 11. Potongan kode *query* SQLite setelah mitigasi menggunakan *Parameterized Queries*.

Gambar 10 memperlihatkan keberhasilan *payload* SQL Injection dalam memanipulasi logika autentikasi sehingga sistem memberikan akses tanpa proses verifikasi kredensial yang valid.



Gambar 12. Hasil pengujian login administrator setelah mitigasi, di mana upaya eksploitasi SQL Injection gagal dilakukan

Gambar 11 menunjukkan implementasi parameterized query sebagai mekanisme mitigasi. Pada pendekatan ini, struktur kueri dipisahkan dari data masukan pengguna sehingga seluruh input diperlakukan sebagai data literal. Hasil pengujian ulang membuktikan bahwa payload SQL Injection tidak lagi dapat memengaruhi logika kueri maupun melakukan bypass autentikasi.

Analisis code review mengungkap bahwa kerentanan tersebut disebabkan oleh penggunaan string concatenation pada pembuatan query SQLite dan rendering output tanpa escaping. Framework Flask yang digunakan tidak menerapkan auto-escaping secara default pada template Jinja2 untuk variabel yang di-render. Setelah dilakukan mitigasi keamanan, beberapa perubahan mendasar diterapkan. Untuk mencegah SQL Injection, semua query diubah menggunakan parameterized query dengan sqlite3 module. Sedangkan untuk XSS, seluruh output yang ditampilkan di-escape menggunakan fungsi escape() dari markupsafe atau mengaktifkan autoescape pada Jinja2 [10].

TABEL 1
RINGKASAN KERENTANAN YANG DITEMUKAN

No	Jenis Kerentanan	Lokasi	Tingkat Risiko	Dampak	Tools Pendeteksi
1	SQL Injection	Form login & pengaduan	High	Akses database tidak sah, data bocor	SQLMap, Burp Suite
2	Stored XSS	Form pengaduan	High	Eksekusi script pada halaman admin	OWASP ZAP
3	Reflected XSS	Parameter pencarian	Medium	Phishing & session hijacking	Chrome DevTools
4	Lack of Input Validation	Semua input form	High	Berbagai serangan injeksi	Manual Testing

TABEL 1
PERBANDINGAN HASIL PENGUJIAN SEBELUM DAN SESUDAH MITIGASI

Jenis Kerentanan	Sebelum Mitigasi	Sesudah Mitigasi	Status
SQL Injection	Berhasil	Gagal	Teratasi
Stored XSS	Berhasil	Gagal	Teratasi
Reflected XSS	Berhasil	Gagal	Teratasi
Session Hijacking	Memungkinkan	Tidak memungkinkan	Teratasi

Re-testing yang dilakukan setelah implementasi mitigasi menunjukkan tidak ada lagi kerentanan SQL Injection maupun XSS yang dapat dieksploitasi dengan payload yang sama. OWASP ZAP hanya melaporkan alert informasi rendah yang tidak

berdampak signifikan. Penerapan Web Application Firewall sederhana berbasis Flask-Limiter juga berhasil membatasi jumlah request mencurigakan.

Hasil penelitian ini sejalan dengan temuan Prasetyo et al. (2024) [1] dan Khairul et al. (2025) [16] yang menyatakan bahwa sebagian besar aplikasi web pemerintahan di Indonesia masih rentan terhadap serangan injeksi akibat kurangnya penerapan secure coding practices. Pendekatan security by design yang diterapkan dalam penelitian ini membuktikan bahwa mitigasi kerentanan dapat dilakukan dengan biaya rendah dan tanpa mengubah arsitektur utama aplikasi, sehingga sangat sesuai untuk implementasi di tingkat desa dan instansi kecil [2]. Penerapan parameterized query dan output escaping tidak hanya menyelesaikan masalah SQL Injection dan XSS, tetapi juga meningkatkan kualitas kode secara keseluruhan. Aplikasi menjadi lebih stabil dan mudah dipelihara. Temuan ini menegaskan pentingnya melakukan penetration testing sejak tahap awal pengembangan, bukan hanya sebagai aktivitas setelah aplikasi selesai dibangun.

Secara keseluruhan, penelitian ini berhasil mendemonstrasikan bahwa aplikasi pengaduan masyarakat berskala kecil berbasis Flask dan SQLite dapat diamankan dengan baik apabila pengembang menerapkan prinsip-prinsip keamanan dasar OWASP secara konsisten. Hasil mitigasi yang efektif menunjukkan bahwa kesenjangan antara teori keamanan dan praktik pengembangan di tingkat lokal dapat diatasi dengan pendekatan yang sederhana namun tepat sasaran.

IV. SIMPULAN

Berdasarkan hasil penelitian yang telah dilakukan menggunakan pendekatan Vulnerability Assessment and Penetration Testing (VAPT), aplikasi web pengaduan masyarakat LaporAman teridentifikasi memiliki beberapa kerentanan keamanan yang berisiko terhadap kerahasiaan, integritas, dan ketersediaan data, khususnya kerentanan SQL Injection pada mekanisme autentikasi administrator serta Stored Cross-Site Scripting (XSS) pada fitur pengaduan masyarakat. Hasil vulnerability assessment dan penetration testing menunjukkan bahwa kedua kerentanan tersebut dapat dieksploitasi oleh penyerang untuk memperoleh akses tidak sah ke sistem, memanipulasi data, maupun mengeksekusi skrip berbahaya pada sisi klien yang berpotensi mengakibatkan pencurian informasi sensitif dan penyalahgunaan hak akses administrator. Analisis lebih lanjut melalui proses code review mengungkap bahwa kerentanan SQL Injection disebabkan oleh penggunaan query dinamis yang dibangun menggunakan teknik string concatenation tanpa mekanisme parameterisasi, sedangkan kerentanan Stored XSS terjadi akibat tidak diterapkannya proses sanitasi dan output escaping terhadap data yang berasal dari pengguna. Untuk mengatasi permasalahan tersebut, dilakukan implementasi mitigasi berupa penerapan parameterized query (prepared statement), validasi input, auto-escaping pada template Jinja2, serta penguatan mekanisme pengolahan data yang ditampilkan kepada pengguna. Hasil re-testing setelah mitigasi menunjukkan bahwa payload SQL Injection maupun Stored XSS yang sebelumnya berhasil dieksploitasi tidak lagi dapat memengaruhi logika aplikasi maupun dieksekusi oleh peramban, sehingga tingkat keamanan sistem mengalami peningkatan yang signifikan. Temuan penelitian ini menegaskan bahwa penerapan secure coding practices, pengujian keamanan secara berkala, serta penerapan prinsip security by design sejak tahap perancangan dan pengembangan sistem merupakan langkah yang efektif untuk meminimalkan risiko serangan siber pada aplikasi web. Selain itu, penelitian ini memberikan kontribusi praktis berupa contoh implementasi mitigasi yang sederhana namun efektif dan dapat diterapkan pada aplikasi layanan publik berskala kecil hingga menengah yang dikembangkan menggunakan framework Python dan basis data SQLite, sehingga dapat menjadi referensi bagi pengembang dalam membangun sistem yang lebih aman, andal, dan tahan terhadap berbagai bentuk serangan aplikasi web.

UCAPAN TERIMA KASIH

Penulis mengucapkan terima kasih kepada Bapak Azis, S.Kom., M.Kom. selaku dosen mata kuliah Keamanan Data dan Informasi yang telah memberikan bimbingan, arahan, dan dukungan selama proses penelitian dan penyusunan artikel ini. Penulis juga menyampaikan terima kasih kepada rekan satu tim yang telah berkontribusi dalam proses pengujian, analisis, serta penyelesaian penelitian sehingga artikel ini dapat diselesaikan dengan baik.

DAFTAR PUSTAKA

- [1] K. Imam, M. Hanif, Z. Ubadillah, and N. Azise, "Sistem Informasi Pelayanan Pengaduan Masyarakat Berbasis Website," 2025.
- [2] M. Darip, "DESAIN DAN IMPLEMENTASI SISTEM APLIKASI PENGADUAN MASYARAKAT BERBASIS WEB," *INFOTECH journal*, vol. 11, no. 1, pp. 46–53, Feb. 2025, doi: 10.31949/infotech.v11i1.12976.
- [3] Annas Muhammad, Adek Rizal, and Affrilia Yesy, "Web Application Firewall (WAF) Design to Detect and Aticipate Hacking in Web-Based Application," 2024.
- [4] N. A. Prasetyo, R. B. Huwae, and A. H. Jatmika, "AUDIT DAN ANALISIS WEBSITE PEMERINTAH MENGGUNAKAN PENGUJIAN PENETRASI SQL INJECTION DAN CROSS SITE SCRIPTING (XSS) (Audit and Analysis of Government Websites Using SQL Injection and Cross-Site Scripting (XSS) Penetration Testing)," 2024. [Online]. Available: <http://jtika.if.unram.ac.id/index.php/JTIKA/>
- [5] A. Hannousse, S. Yahiouche, and M. C. Nait-Hamoud, "Twenty-two years since revealing cross-site scripting attacks: a systematic mapping and a comprehensive survey," May 2022, doi: 10.1016/j.cosrev.2024.100634.
- [6] A. Fadlil, I. Riadi, and M. A. Mu'Min, "Mitigation from SQL Injection Attacks on Web Server using Open Web Application Security Project Framework," *International Journal of Engineering, Transactions A: Basics*, vol. 37, no. 4, pp. 635–645, Apr. 2024, doi: 10.5829/ije.2024.37.04a.06.

- [7] M. A. Z. Risky and Y. Yuhandri, "Optimalisasi dalam Penetrasi Testing Keamanan Website Menggunakan Teknik SQL Injection dan XSS," *Jurnal Sistim Informasi dan Teknologi*, pp. 215–220, Aug. 2021, doi: 10.37034/jsisfotek.v3i4.68.
- [8] M. Irfa'issurur and B. P. Josaphat, "Machine Learning for Cybersecurity: Web Attack Detection (Brute Force, XSS, SQL Injection)," *InPrime: Indonesian Journal of Pure and Applied Mathematics*, vol. 7, no. 1, pp. 1–15, Feb. 2025, doi: 10.15408/inprime.v7i1.41025.
- [9] Josua Karlos Manuel and Rezki Kurniati, "Analisis Keamanan Website E-Pinter Terhadap Serangan SQL Injection dan XSS," *Jurnal Publikasi Teknik Informatika*, vol. 4, no. 3, pp. 46–60, Aug. 2025, doi: 10.55606/jupti.v4i3.5234.
- [10] Adi Suryadin, Latif Kumiadin, Lilik Widyawati, Raisul Azhar, and Muhammad Azwar, "Evaluasi Penerapan Pertahanan Proaktif Waf dan Hids Terhadap Eksploitasi Kerentanan Aplikasi Web," 2025.
- [11] A. D. Yudistiawan and N. Anwar, "Website Application Security Value Analysis Using Crawling Method Against SQL Injection Attacks," *Mobile and Forensics*, vol. 6, no. 1, pp. 39–50, Aug. 2024, doi: 10.12928/mf.v6i1.8198.
- [12] L. A. Nugraha, I. A. Kautsar, and A. S. Fitriani, "SQL Injection: Analisis Efektivitas Uji Penetrasi dalam Aplikasi Web," *SMATIKA JURNAL*, vol. 14, no. 01, pp. 111–123, Jun. 2024, doi: 10.32664/smatika.v14i01.1224.
- [13] H. Kasdana *et al.*, "Analisis Kerentanan Aplikasi Web terhadap SQL Injection dan XSS Berdasarkan Survei Pengguna dengan Kategori Risiko," 2025. [Online]. Available: <https://jti.aisyahuniversity.ac.id/index.php/AJIEE>
- [14] N. Alfi, S. Purba, M. Ula, and S. Fachrurrazi, "Aplikasi Sistem Informasi Pengaduan Masyarakat Berbasis Website Pada Dinas Komunikasi Informatika Dan Persandian Kota Lhokseumawe," 2024, [Online]. Available: <https://github.com/twbs/bootstrap>
- [15] J. R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," *Sci. Rep.*, vol. 14, no. 1, Dec. 2024, doi: 10.1038/s41598-023-48845-4.
- [16] Khairul, A. Abdullah, and S. Sucipto, "Penetration Testing pada Kerentanan Keamanan Sistem PELAKAT Menggunakan SQL Injection," *Jurnal Nasional Teknologi dan Sistem Informasi*, vol. 11, no. 1, pp. 78–86, May 2025, doi: 10.25077/teknosi.v11i1.2025.78-86.